

Regularization and Ensemble Methods

LASSO, Ridge, Elastic Net, Data Augmentation, Noise Injection, Early Stopping,
Sparse Representations, Dropout, Bagging and Ensembles

A mathematical and practical guide with derivations, examples, and visual intuition

Unifying question: how do we deliberately constrain, perturb, average, or stop a learning algorithm so that it generalizes better to unseen data?

Contents

1	The unifying idea: regularization	4
1.1	Bias–variance viewpoint	4
2	Baseline: ordinary least squares	5
3	Ridge regression: L2 regularization	5
3.1	Objective	5
3.2	Closed-form solution	5
3.3	Spectral interpretation	6
3.4	Constrained form	6
3.5	Worked orthogonal-design example	7
3.6	Coefficient path	7
3.7	Ridge in neural networks	7
4	LASSO: L1 regularization	8
4.1	Objective	8
4.2	Why exact zeros occur	8
4.3	Soft thresholding: the key calculation	8
4.4	Coordinate-descent view	9
4.5	Correlated predictors	9
4.6	Bayesian interpretation	10
5	Elastic Net: combining L1 and L2	10
5.1	What each term contributes	10
5.2	Example with two correlated predictors	11
5.3	Hyperparameter interpretation	11
6	LASSO vs Ridge vs Elastic Net	12
7	Regularization beyond penalties	12
8	Dataset augmentation	12
8.1	Basic idea	12
8.2	What augmentation teaches	13
8.3	Examples by data type	13
8.4	Augmentation as prior knowledge	14
8.5	Augmentation versus simply duplicating data	14
9	Noise injection	14
9.1	Input noise	14
9.2	Small-noise Taylor expansion	14
9.3	Exact linear-regression connection	14
9.4	Other locations for noise	15
9.5	Choosing the noise scale	15
10	Early stopping	15
10.1	Algorithm	15
10.2	Why stopping early regularizes	16
10.3	Linear least-squares spectral derivation	16

10.4	Patience and minimum improvement	17
10.5	Leakage warning	17
11	Sparse representations	17
11.1	Sparse coding	17
11.2	Why sparse representations can help	17
11.3	Sparse autoencoder idea	17
12	Dropout	18
12.1	Mechanism	18
12.2	Numerical example	18
12.3	Why dropout regularizes	19
12.4	Dropout rate	19
12.5	Where to use dropout	19
12.6	Dropout versus LASSO	19
13	Bagging	20
13.1	Bootstrap sampling	20
13.2	Aggregation	20
13.3	Variance reduction derivation	21
13.4	Why decision trees benefit strongly	21
14	Random forests as bagging plus feature randomization	21
15	Out-of-bag evaluation	22
16	General ensemble methods	22
16.1	Simple averaging	22
16.2	Weighted averaging	22
16.3	Optimal linear combination and covariance	22
16.4	Stacking	23
16.5	Bagging versus boosting	23
17	How the methods relate mathematically	23
18	Detailed worked comparison on one hypothetical problem	23
18.1	Ridge approach	24
18.2	LASSO approach	24
18.3	Elastic Net approach	24
18.4	Neural-network approach	24
18.5	Ensemble approach	24
19	Regularization in neural-network optimization	24
20	Interaction effects and common mistakes	25
20.1	Too much regularization	25
20.2	Using augmentation that changes the label	25
20.3	Applying LASSO without scaling	25
20.4	Selecting hyperparameters on the test set	25
20.5	Random bootstrap for time series	25
20.6	Confusing parameter sparsity with dropout	25

21 Diagnostic patterns	25
21.1 High training performance, weak validation performance	25
21.2 Training and validation both weak	26
21.3 LASSO solutions unstable across folds	26
21.4 Ensemble gives little improvement	26
22 Practical selection guide	26
23 Connection to bias, variance, and model capacity	26
24 Compact mathematical summary	27
25 Final conceptual map	28
Suggested foundational references	29

1. The unifying idea: regularization

Suppose a model is parameterized by θ and trained on observations $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$. Empirical risk minimization chooses

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_{\theta}(x_i)).$$

A sufficiently flexible model can often lower this training objective by learning not only stable structure but also accidental structure: sampling noise, idiosyncratic outliers, nuisance variation, or spurious interactions.

Regularization deliberately makes some solutions harder to obtain than others.

A useful general form is

$$\hat{\theta} = \arg \min_{\theta} \left[\underbrace{\frac{1}{n} \sum_{i=1}^n \ell_i(\theta)}_{\text{fit to observed data}} + \underbrace{\lambda \Omega(\theta)}_{\text{explicit regularization}} \right].$$

But not all regularization appears as an explicit penalty. Early stopping, data augmentation, noise injection, dropout, and ensemble averaging all regularize through the *training procedure or prediction mechanism*.

1.1. Bias–variance viewpoint

For squared-error prediction, a classical decomposition at a fixed point x is

$$\mathbb{E}_{\mathcal{D}, \varepsilon} \left[(Y - \hat{f}_{\mathcal{D}}(x))^2 \right] = \underbrace{\sigma_{\varepsilon}^2}_{\text{irreducible noise}} + \underbrace{\left(\mathbb{E}_{\mathcal{D}} \hat{f}_{\mathcal{D}}(x) - f(x) \right)^2}_{\text{bias}^2} + \underbrace{\text{Var}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}(x))}_{\text{variance}}.$$

Many regularizers accept a small increase in bias in exchange for a larger decrease in variance.

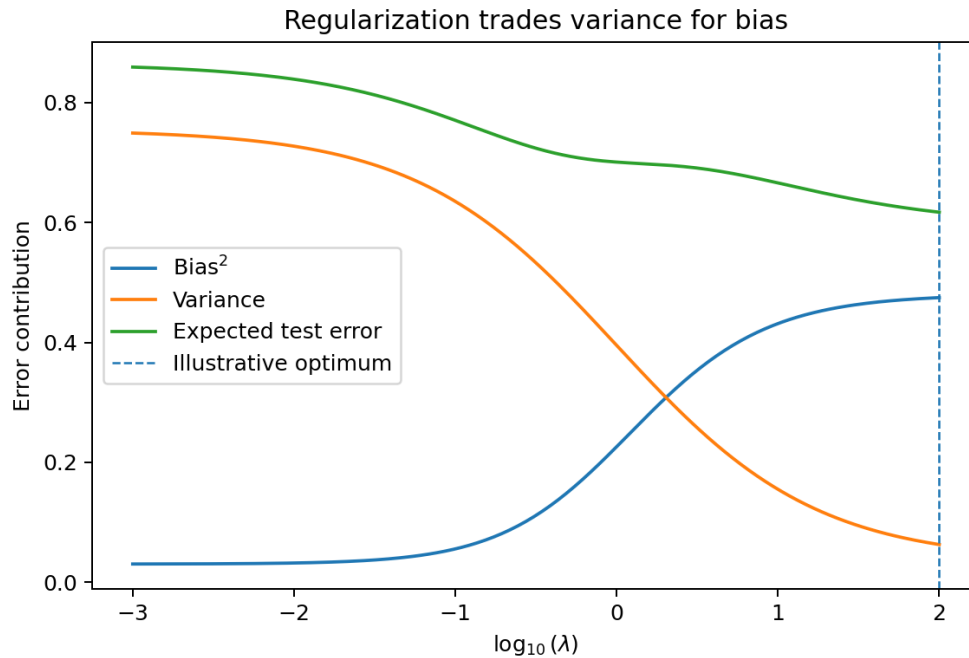


Figure 1: Illustrative bias–variance tradeoff as regularization strength increases.

Mental model. Ridge/LASSO/Elastic Net constrain parameters. Augmentation and noise modify the effective training distribution. Early stopping constrains optimization time. Dropout trains stochastic subnetworks. Bagging reduces prediction variance by averaging models.

2. Baseline: ordinary least squares

Many regularization ideas are clearest in linear regression. Let

$$y = X\beta + \varepsilon, \quad X \in \mathbb{R}^{n \times p}, \quad \beta \in \mathbb{R}^p.$$

Ordinary least squares solves

$$\hat{\beta}_{\text{OLS}} = \arg \min_{\beta} \|y - X\beta\|_2^2.$$

If $X^T X$ is invertible,

$$\boxed{\hat{\beta}_{\text{OLS}} = (X^T X)^{-1} X^T y.}$$

When predictors are highly correlated, $X^T X$ can be poorly conditioned. Small changes in the data can then produce large changes in $\hat{\beta}$, which is a variance problem. If $p \geq n$, infinitely many exact interpolating solutions may exist.

Throughout the regularized-regression sections, predictors should generally be standardized before applying penalties because the penalty acts on coefficient magnitudes. Without standardization, measurement units influence the effective amount of shrinkage.

3. Ridge regression: L2 regularization

3.1. Objective

Ridge regression solves

$$\boxed{\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \left[\|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \right],}$$

where

$$\|\beta\|_2^2 = \sum_{j=1}^p \beta_j^2, \quad \lambda \geq 0.$$

Usually the intercept is not penalized.

The loss rewards fit. The penalty rewards small coefficients. Larger λ means stronger shrinkage.

3.2. Closed-form solution

Differentiate:

$$\frac{\partial}{\partial \beta} [(y - X\beta)^T (y - X\beta) + \lambda \beta^T \beta] = -2X^T (y - X\beta) + 2\lambda \beta.$$

Set this equal to zero:

$$X^T X \beta + \lambda \beta = X^T y.$$

Therefore

$$\boxed{\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y.}$$

Ridge effectively raises all eigenvalues of $X^T X$ by λ , improving conditioning.

3.3. Spectral interpretation

Take the singular value decomposition

$$X = UDV^T,$$

with singular values d_j . Then

$$\hat{\beta}_{\text{ridge}} = V \operatorname{diag} \left(\frac{d_j}{d_j^2 + \lambda} \right) U^T y.$$

OLS has factors $1/d_j$. Ridge replaces these by

$$\frac{d_j}{d_j^2 + \lambda}.$$

Directions associated with small singular values are shrunk most aggressively. Those are precisely the unstable directions created by multicollinearity.

3.4. Constrained form

Ridge is equivalent, for an appropriate one-to-one correspondence between λ and t , to

$$\min_{\beta} \|y - X\beta\|_2^2 \quad \text{subject to} \quad \|\beta\|_2^2 \leq t.$$

The feasible region is a sphere (a circle in two dimensions).

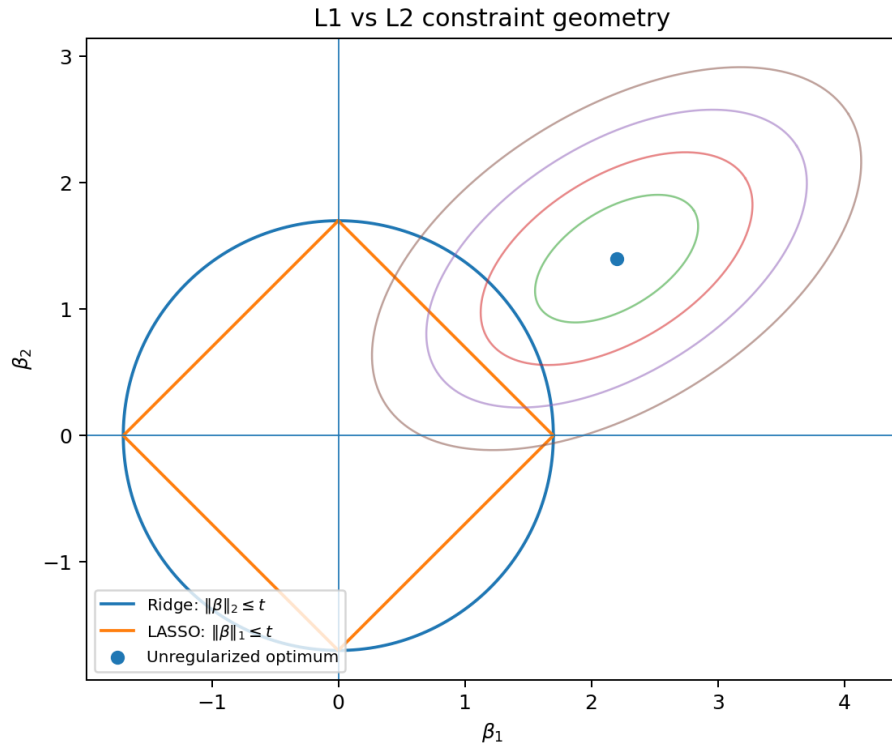


Figure 2: L2 and L1 constraint geometry. L1 corners are central to LASSO sparsity.

3.5. Worked orthogonal-design example

Assume standardized orthogonal columns with

$$X^T X = I.$$

Then OLS is

$$\hat{\beta}_{\text{OLS}} = X^T y.$$

Ridge becomes

$$\hat{\beta}_{\text{ridge}} = (I + \lambda I)^{-1} X^T y = \frac{1}{1 + \lambda} \hat{\beta}_{\text{OLS}}.$$

If

$$\hat{\beta}_{\text{OLS}} = (3, -2, 0.5)^T, \quad \lambda = 1,$$

then

$$\hat{\beta}_{\text{ridge}} = (1.5, -1, 0.25)^T.$$

All coefficients are continuously shrunk toward zero, but none are forced exactly to zero.

3.6. Coefficient path

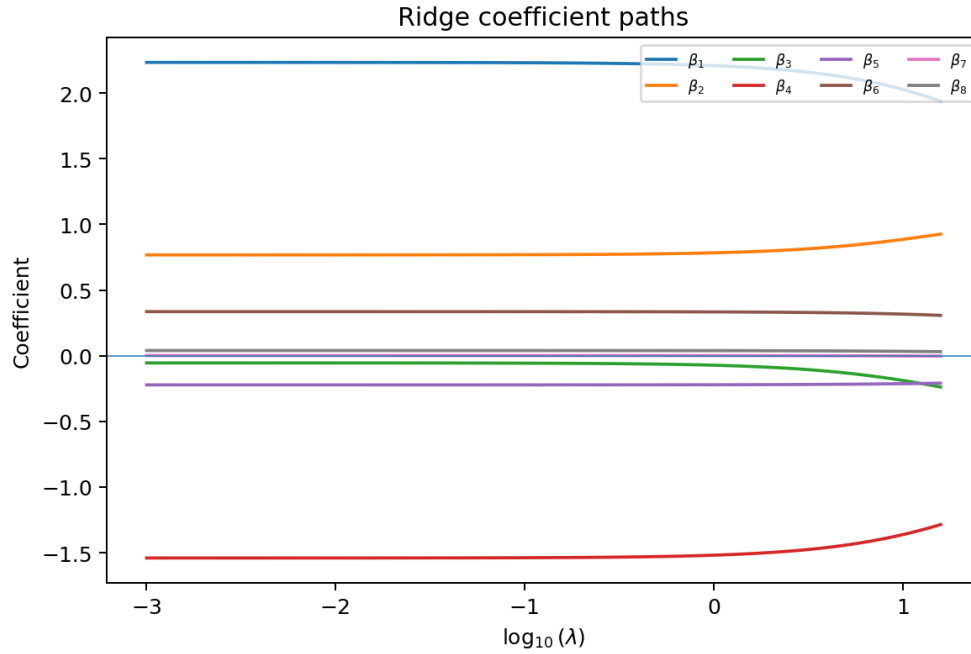


Figure 3: Illustrative Ridge coefficient paths. Increasing λ smoothly contracts all coefficients.

3.7. Ridge in neural networks

An L2 penalty on a network's weights is

$$L_{\text{total}} = L_{\text{data}} + \frac{\lambda}{2} \sum_l \|W^{[l]}\|_F^2.$$

The gradient becomes

$$\frac{\partial L_{\text{total}}}{\partial W} = \frac{\partial L_{\text{data}}}{\partial W} + \lambda W.$$

With vanilla SGD,

$$W_{t+1} = W_t - \eta(g_t + \lambda W_t) = (1 - \eta\lambda)W_t - \eta g_t.$$

Thus each step contains a multiplicative shrinkage term. This motivates the phrase **weight decay**. With adaptive optimizers such as Adam, decoupled weight decay (AdamW) is usually conceptually cleaner than simply adding an L2 term to the loss because adaptive preconditioning otherwise also rescales the regularization gradient.

4. LASSO: L1 regularization

4.1. Objective

LASSO solves

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \left[\frac{1}{2} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right],$$

where

$$\|\beta\|_1 = \sum_{j=1}^p |\beta_j|.$$

Its defining property is that it can produce

$$\hat{\beta}_j = 0$$

exactly.

4.2. Why exact zeros occur

Unlike β^2 , the function $|\beta|$ has a kink at zero. Its subgradient is

$$\partial |\beta| = \begin{cases} \{1\}, & \beta > 0, \\ [-1, 1], & \beta = 0, \\ \{-1\}, & \beta < 0. \end{cases}$$

At zero, a range of loss gradients can be balanced by the L1 penalty without moving the coefficient away from zero.

Geometrically, the L1 constraint

$$\sum_j |\beta_j| \leq t$$

forms a diamond in two dimensions. Smooth elliptical loss contours frequently first touch this feasible region at a corner, and corners lie on coordinate axes. Hence some coefficients are exactly zero.

4.3. Soft thresholding: the key calculation

With a single standardized orthogonal predictor, the LASSO problem reduces to

$$\min_{\beta} \frac{1}{2} (\beta - z)^2 + \lambda |\beta|,$$

where z is the unregularized coefficient. The solution is

$$\hat{\beta} = \text{soft}(z, \lambda) = \text{sign}(z) \max(|z| - \lambda, 0).$$

This is called **soft thresholding**.

For $z = 2.4$ and $\lambda = 0.7$,

$$\hat{\beta} = + \max(2.4 - 0.7, 0) = 1.7.$$

For $z = 0.5$,

$$\hat{\beta} = \max(0.5 - 0.7, 0) = 0.$$

For $z = -1.1$,

$$\hat{\beta} = - \max(1.1 - 0.7, 0) = -0.4.$$

LASSO therefore performs both **shrinkage** and **variable selection**.

4.4. Coordinate-descent view

For standardized predictors, coordinate descent repeatedly forms a partial residual excluding feature j :

$$r^{(-j)} = y - \sum_{k \neq j} x_k \beta_k.$$

The update is essentially

$$\beta_j \leftarrow \frac{\text{soft}(x_j^T r^{(-j)} / n, \lambda)}{x_j^T x_j / n}.$$

This is why coordinate descent is particularly natural for L1 objectives.

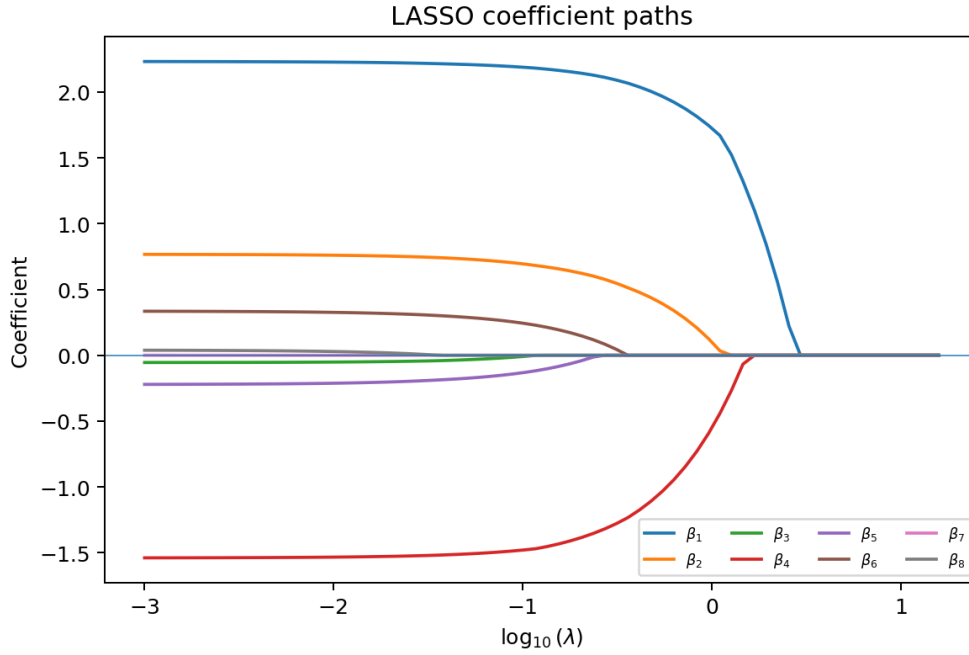


Figure 4: Illustrative LASSO coefficient paths. Coefficients hit exactly zero as regularization increases.

4.5. Correlated predictors

Suppose two predictors x_1, x_2 are almost duplicates and both predict y . LASSO often has little incentive to retain both. It may select one and set the other to zero. Which one survives can be unstable under small sample perturbations.

That behavior can be excellent when genuine feature selection is desired, but undesirable when strongly correlated variables should be retained as a group. This is one motivation for Elastic Net.

4.6. Bayesian interpretation

Ridge corresponds to a Gaussian prior on coefficients:

$$\beta_j \sim \mathcal{N}(0, \tau^2).$$

LASSO corresponds to a Laplace (double-exponential) prior:

$$p(\beta_j) \propto \exp(-\lambda |\beta_j|).$$

The regularized estimator can be interpreted as a maximum a posteriori estimate. The sharper peak of the Laplace density around zero helps explain the stronger preference for sparse coefficients.

5. Elastic Net: combining L1 and L2

Elastic Net combines the two penalties:

$$\hat{\beta}_{\text{EN}} = \arg \min_{\beta} \left[\frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \left(\alpha \|\beta\|_1 + \frac{1-\alpha}{2} \|\beta\|_2^2 \right) \right].$$

Here

$$0 \leq \alpha \leq 1.$$

When $\alpha = 1$, this is LASSO. When $\alpha = 0$, it is Ridge up to parameterization conventions.

5.1. What each term contributes

The L1 component provides sparsity:

$$\beta_j \rightarrow 0 \quad \text{and possibly} \quad \beta_j = 0.$$

The L2 component stabilizes solutions in the presence of correlated predictors and tends to keep groups of correlated variables together more readily than pure LASSO.

In a standardized coordinate update, one obtains the rough structure

$$\beta_j \leftarrow \frac{\text{soft}(z_j, \lambda\alpha)}{1 + \lambda(1 - \alpha)}.$$

The numerator applies L1 thresholding; the denominator applies L2 shrinkage.

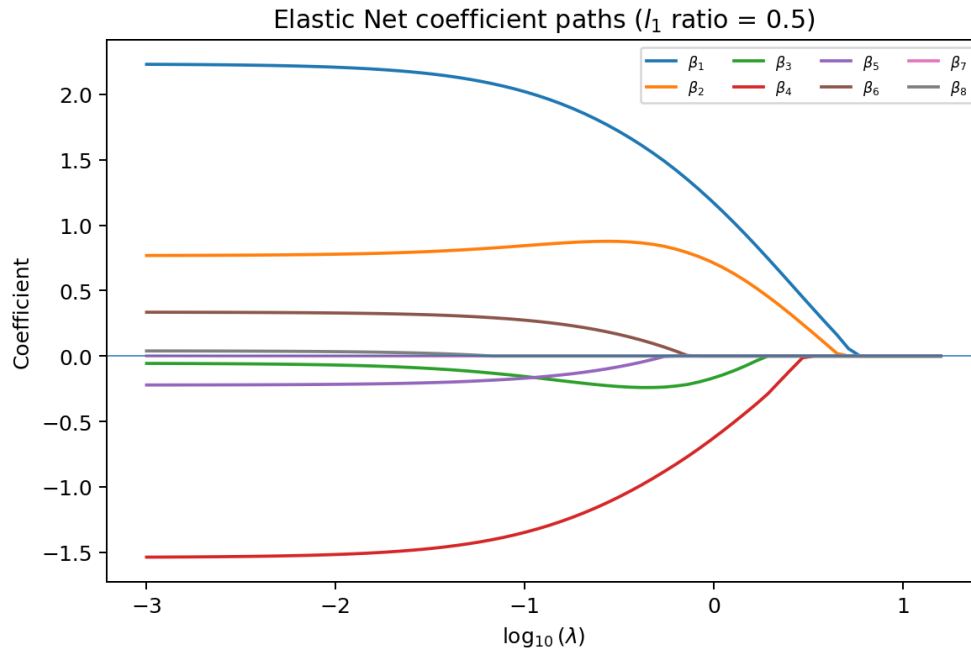


Figure 5: Illustrative Elastic Net coefficient paths. It combines zeroing behavior with smoother grouped shrinkage.

5.2. Example with two correlated predictors

Assume x_1 and x_2 have correlation 0.95, and both measure approximately the same latent signal. A pure LASSO solution might be

$$(\hat{\beta}_1, \hat{\beta}_2) = (1.8, 0).$$

A Ridge solution might be

$$(1.0, 0.9).$$

An Elastic Net solution might be

$$(1.15, 0.75),$$

while also setting several unrelated weak predictors exactly to zero. The numbers are illustrative, but the behavior is characteristic.

5.3. Hyperparameter interpretation

There are two distinct questions:

λ : How much total regularization?

α : How LASSO-like versus Ridge-like?

They should generally be selected inside a proper validation procedure. For time series, use temporally ordered validation rather than random cross-validation.

6. LASSO vs Ridge vs Elastic Net

Property	Ridge	LASSO	Elastic Net
Penalty	$\lambda \sum \beta_j^2$	$\lambda \sum \beta_j $	L1 + L2
Exact zeros	Usually no	Yes	Yes
Correlated predictors	Shares weight	Often selects one	Often groups better
Stability	High	Can be unstable under collinearity	Often intermediate/high
Feature selection	No	Yes	Yes
Closed form	Yes for linear least squares	No general closed form	No general closed form
Best intuition	Continuous shrinkage	Sparse selection	Sparse + stabilized selection

Table 1: Core comparison of the three regularized linear estimators.

If many weak predictors each carry a little signal, Ridge is often attractive. If only a small subset is expected to matter and interpretability matters, LASSO is attractive. If features are numerous, correlated, and some sparsity is still desired, Elastic Net is often the safer default.

7. Regularization beyond penalties

Explicit coefficient penalties are only one strategy. A broader view is

generalization control = constraint on solutions, data, optimization, representation, or prediction averaging.

The next methods can all be understood through this lens.

8. Dataset augmentation

8.1. Basic idea

Dataset augmentation creates additional training examples by applying transformations that should preserve the target. Let

$$T \sim q(T)$$

be a random transformation. Instead of minimizing only

$$\frac{1}{n} \sum_i \ell(y_i, f_\theta(x_i)),$$

we minimize the augmented risk

$$L_{\text{aug}}(\theta) = \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{T \sim q} [\ell(y_i, f_\theta(Tx_i))].$$

In practice, the expectation is approximated by randomly sampling transformations during training.

8.2. What augmentation teaches

Suppose an image of a dog remains a dog under a small translation. We would like

$$f(x) \approx f(Tx).$$

Augmentation creates training pressure toward precisely this invariance.

The essential assumption is therefore not “more data is always good.” It is

$$y(Tx) = y(x) \quad \text{for the chosen transformations.}$$

If this assumption is false, augmentation creates mislabeled data.

Augmentation expands each observation into a local invariance cloud

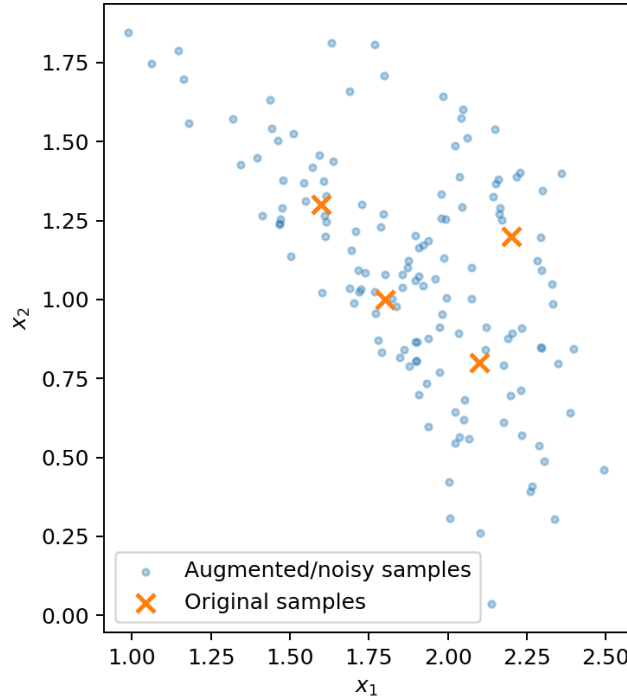


Figure 6: A 2D analogy: augmentation expands each observation into a local cloud of target-preserving variants.

8.3. Examples by data type

For images, common target-preserving transformations can include crops, translations, small rotations, flips when orientation is irrelevant, color jitter, random erasing, Mixup, or CutMix.

For audio, examples include time shifts, small speed changes, background noise, time/frequency masking, or room-response perturbations.

For text, augmentation is much more delicate because small token changes can alter semantics. Back-translation, controlled paraphrasing, masking, or model-generated variants can be useful only when label preservation is credible.

For tabular/financial data, arbitrary perturbation is especially dangerous. A synthetic observation must respect economic and structural constraints. Time reversal, random row shuffling, or unconstrained Gaussian perturbation can easily destroy the data-generating process.

8.4. Augmentation as prior knowledge

Augmentation inserts a prior about symmetry or invariance directly into training. If G is a transformation group and the target should be invariant under $g \in G$, augmentation encourages

$$f(gx) = f(x).$$

This is a powerful form of regularization because it does not merely prefer small parameters; it says what kinds of changes in the input should *not* change the prediction.

8.5. Augmentation versus simply duplicating data

Duplicating an observation exactly changes almost nothing if the objective is an average: the duplicated point just receives extra weight. Useful augmentation creates *new variants* that encode desired invariance.

9. Noise injection

Noise injection is closely related to augmentation but emphasizes random perturbations rather than semantically defined transformations.

9.1. Input noise

Train on

$$\tilde{x} = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I),$$

by minimizing

$$L_{\text{noise}}(\theta) = \mathbb{E}_{\varepsilon} [\ell(y, f_{\theta}(x + \varepsilon))].$$

This encourages the prediction to be locally stable around observed points.

9.2. Small-noise Taylor expansion

For a smooth scalar prediction $f(x)$, expand around x :

$$f(x + \varepsilon) \approx f(x) + \nabla_x f(x)^T \varepsilon + \frac{1}{2} \varepsilon^T H_f(x) \varepsilon.$$

For zero-mean isotropic noise,

$$\mathbb{E}[\nabla f^T \varepsilon] = 0.$$

The expected noisy objective therefore introduces terms related to local derivatives. Intuitively, models with highly sensitive input-output mappings are penalized because noise causes their loss to rise strongly.

9.3. Exact linear-regression connection

Take linear regression

$$f_{\beta}(x) = x^T \beta$$

and inject independent Gaussian noise into the predictors:

$$\tilde{X} = X + E, \quad E_{ij} \sim \mathcal{N}(0, \sigma^2).$$

Consider the expected squared error:

$$\mathbb{E}_E \|y - (X + E)\beta\|_2^2.$$

Expand:

$$\|y - X\beta - E\beta\|_2^2.$$

The cross term vanishes in expectation because $\mathbb{E}[E] = 0$. Also,

$$\mathbb{E} \|E\beta\|_2^2 = n\sigma^2 \|\beta\|_2^2.$$

Therefore

$$\boxed{\mathbb{E}_E \|y - (X + E)\beta\|_2^2 = \|y - X\beta\|_2^2 + n\sigma^2 \|\beta\|_2^2.}$$

So, in this setting, Gaussian input noise is exactly equivalent in expectation to an L2 penalty. This is a beautiful example of how seemingly different regularization mechanisms can be mathematically connected.

9.4. Other locations for noise

Noise can be injected into inputs, hidden activations, weights, gradients, or labels. These are not interchangeable.

Activation noise encourages internal representations to remain useful despite perturbation. Weight noise smooths dependence on parameter values. Gradient noise changes optimization dynamics. Label noise is generally dangerous unless it is deliberate label smoothing or based on a valid uncertainty model.

9.5. Choosing the noise scale

If σ is too small, the regularization effect is negligible. If too large, the perturbed sample can cross into a different semantic class or an implausible region of feature space. Feature scaling is therefore crucial: a single global σ can be meaningless when columns have different units.

10. Early stopping

10.1. Algorithm

During training, evaluate a validation metric after each epoch (or after a chosen number of updates). Keep the parameters from the best validation point and stop once performance has failed to improve for a chosen patience window.

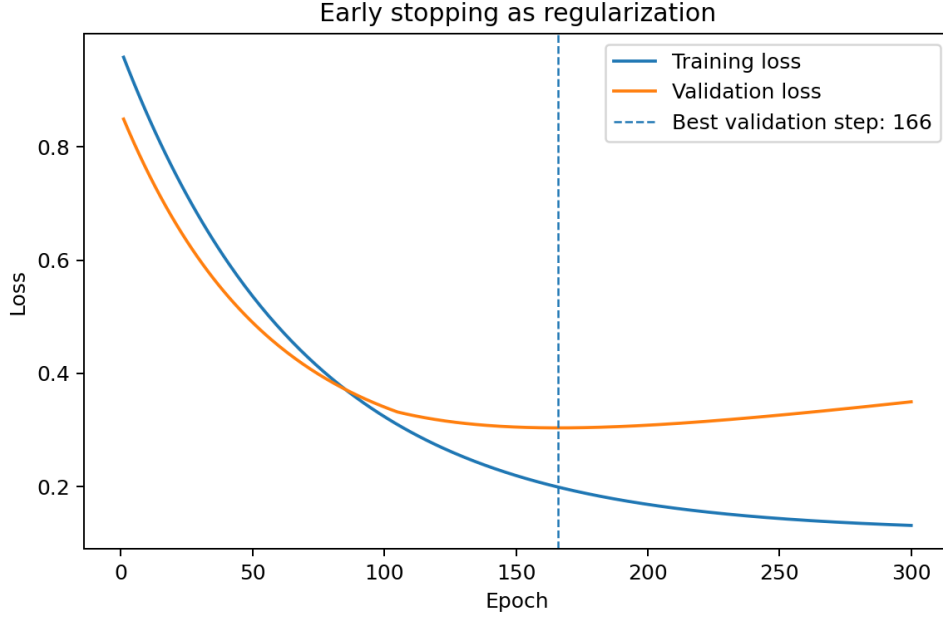


Figure 7: Training loss can keep falling after validation performance starts deteriorating. Early stopping retains an earlier solution.

10.2. Why stopping early regularizes

Optimization time itself controls effective model complexity. Early in training, gradient methods often learn high-signal, relatively smooth directions. Given enough iterations, a flexible model can increasingly fit weak or noisy components.

Early stopping restricts the reachable parameter state:

$$\theta_0 \rightarrow \theta_1 \rightarrow \cdots \rightarrow \theta_T,$$

and chooses a finite T before full empirical-risk minimization.

10.3. Linear least-squares spectral derivation

Consider squared loss

$$L(\beta) = \frac{1}{2} \|y - X\beta\|^2,$$

with gradient descent

$$\beta_{t+1} = \beta_t - \eta X^T (X\beta_t - y).$$

Starting at $\beta_0 = 0$, use the SVD $X = UDV^T$. One can show

$$\beta_t = V \operatorname{diag} \left(\frac{1 - (1 - \eta d_j^2)^t}{d_j} \right) U^T y.$$

Compare this with OLS:

$$\beta_{\text{OLS}} = V \operatorname{diag}(1/d_j) U^T y.$$

The multiplicative filter

$$1 - (1 - \eta d_j^2)^t$$

approaches one quickly for large singular values but slowly for small singular values. Early stopping therefore suppresses poorly identified directions, similarly in spirit to Ridge.

10.4. Patience and minimum improvement

A practical stopping rule may track validation loss V_t and define an improvement only if

$$V_t < V_{\text{best}} - \delta,$$

where δ is a minimum improvement threshold. Stop if no improvement occurs for P checkpoints.

Patience P protects against noisy validation curves. A very small patience can stop because of random fluctuation; a very large patience weakens the regularization effect.

10.5. Leakage warning

The validation set used for early stopping is part of model selection. If you repeatedly inspect the final test set to decide when to stop, the test set is no longer a clean test set. For time-series problems, the early-stopping validation block must also respect temporal order and any necessary embargo.

11. Sparse representations

Parameter sparsity and representation sparsity are different ideas.

11.1. Sparse coding

Suppose an input $x \in \mathbb{R}^d$ is represented using a dictionary $D \in \mathbb{R}^{d \times k}$ and code $a \in \mathbb{R}^k$:

$$x \approx Da.$$

Sparse coding solves

$$\hat{a} = \arg \min_a \left[\frac{1}{2} \|x - Da\|_2^2 + \lambda \|a\|_1 \right].$$

The representation a has many zeros. The dictionary can be overcomplete, with $k > d$, while each observation uses only a small subset of dictionary atoms.

11.2. Why sparse representations can help

Sparsity can provide interpretability, robustness, efficient storage, feature selection at the representation level, and a compositional view in which each example activates only a few latent factors.

11.3. Sparse autoencoder idea

A neural network can explicitly encourage hidden units to be rarely active. If $\hat{\rho}_j$ is the empirical average activation of hidden unit j and ρ is a small target activity, one can add a KL divergence penalty

$$\sum_j \text{KL}(\rho \parallel \hat{\rho}_j),$$

where

$$\text{KL}(\rho \parallel \hat{\rho}) = \rho \log \frac{\rho}{\hat{\rho}} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}}.$$

This encourages units to specialize and remain inactive on most examples.

12. Dropout

Dropout is related to sparse activations during training but should not be equated with classical sparse representation learning.

12.1. Mechanism

Let hidden activations be

$$h = \phi(Wx + b).$$

With keep probability $q = 1 - p$, draw a mask

$$m_j \sim \text{Bernoulli}(q).$$

Under **inverted dropout**, train with

$$\tilde{h} = \frac{m \odot h}{q}.$$

Each unit is zeroed with probability p .

Because

$$\mathbb{E}[m_j] = q,$$

we have

$$\mathbb{E}\left[\frac{m_j h_j}{q}\right] = h_j.$$

Thus no rescaling is needed at test time: simply use the full network without dropout.

12.2. Numerical example

Suppose

$$h = (2, 1, -3, 4), \quad p = 0.5, \quad q = 0.5.$$

If the sampled mask is

$$m = (1, 0, 1, 0),$$

then

$$\tilde{h} = \frac{(2, 0, -3, 0)}{0.5} = (4, 0, -6, 0).$$

A different mini-batch can receive a different mask. Training therefore sees a huge collection of subnetworks sharing parameters.

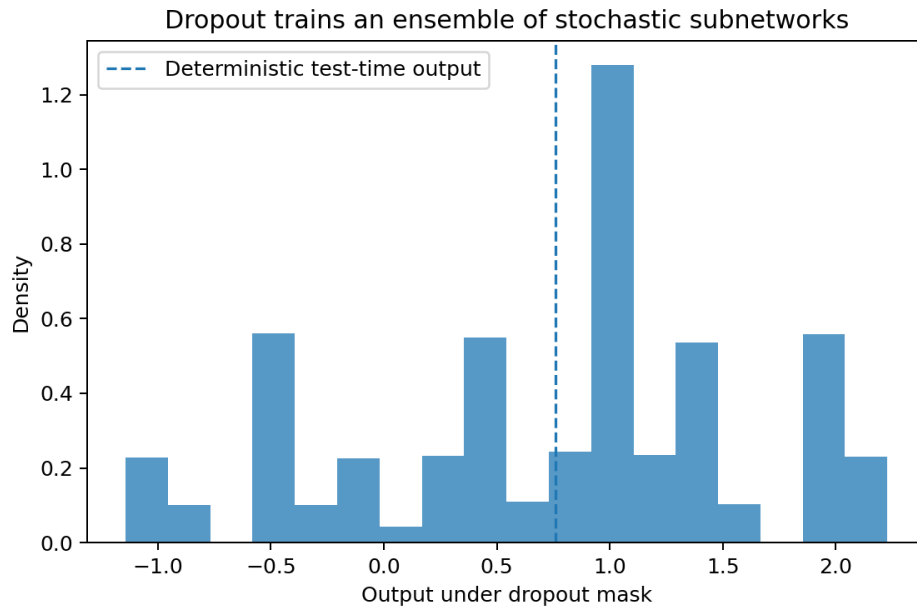


Figure 8: Repeated dropout masks create a distribution of stochastic network outputs around the deterministic test-time computation.

12.3. Why dropout regularizes

Three complementary intuitions are useful.

First, dropout reduces **co-adaptation**: one hidden unit cannot safely depend on another unit always being present.

Second, it approximates training a large ensemble of related subnetworks that share weights.

Third, it injects multiplicative noise into internal representations, forcing predictions to be robust to missing features.

12.4. Dropout rate

A dropout rate $p = 0.5$ in a hidden layer means half of units are removed on average per training pass. This can be strong regularization. Modern architectures frequently use smaller rates such as 0.1–0.3, but the correct rate is architecture- and data-dependent.

Too much dropout creates underfitting and noisy optimization. Too little may have negligible effect.

12.5. Where to use dropout

Dropout is typically active only during training. It is usually placed on hidden activations or specific architectural components. Applying dropout blindly after every layer is not universally beneficial; convolutional, recurrent, residual, and transformer architectures each have conventions that reflect their structure.

12.6. Dropout versus LASSO

LASSO changes the optimization objective so parameters may become exactly zero. Dropout does not usually make weights permanently zero. Instead, it creates temporary zeros in activations (or other tensors) during training. The final test-time model normally uses all units.

13. Bagging

Bagging stands for **bootstrap aggregating**. It regularizes prediction by averaging independently trained models fitted to perturbed datasets.

13.1. Bootstrap sampling

Given n observations, draw n samples *with replacement*. Some original observations occur multiple times, while others are omitted.

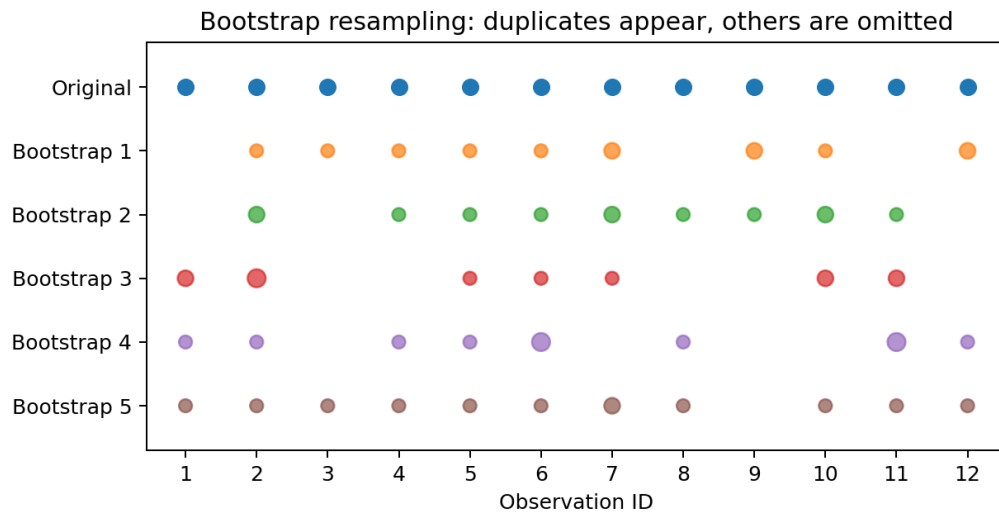


Figure 9: Bootstrap samples contain duplicates and omit some observations. Marker size indicates multiplicity.

For any particular observation, the probability it is absent from one bootstrap sample is

$$\left(1 - \frac{1}{n}\right)^n \rightarrow e^{-1} \approx 0.368.$$

Therefore roughly 36.8% of observations are out-of-bag for a given bootstrap model, while roughly 63.2% unique observations appear at least once.

13.2. Aggregation

Train M models

$$\hat{f}_1(x), \dots, \hat{f}_M(x).$$

For regression, average:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{M} \sum_{m=1}^M \hat{f}_m(x).$$

For classification, average class probabilities or use majority voting.

13.3. Variance reduction derivation

Assume each model prediction has variance σ^2 , and pairwise prediction errors have common correlation ρ . Then

$$\text{Var} \left(\frac{1}{M} \sum_{m=1}^M \hat{f}_m \right) = \frac{1}{M^2} [M\sigma^2 + M(M-1)\rho\sigma^2].$$

Therefore

$$\text{Var}(\bar{f}) = \sigma^2 \left[\rho + \frac{1-\rho}{M} \right].$$

Two implications follow immediately:

- Increasing M reduces the independent part of variance, $(1-\rho)/M$.
- The limiting variance as $M \rightarrow \infty$ is $\rho\sigma^2$. Highly correlated models cannot be averaged into a low-variance predictor.

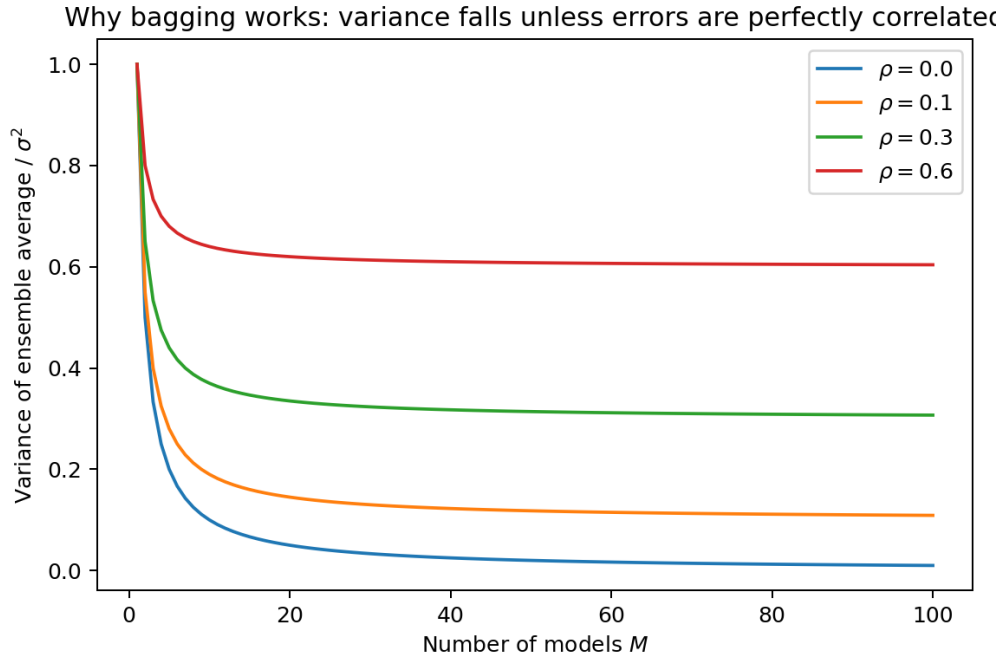


Figure 10: Bagging gains depend on both ensemble size and diversity. Correlated errors create a variance floor.

13.4. Why decision trees benefit strongly

Deep decision trees are high-variance estimators: small changes in training data can alter early splits and produce very different trees. That instability creates exactly the kind of variance that averaging can reduce. This is why bagging trees is particularly effective.

14. Random forests as bagging plus feature randomization

A random forest adds another source of diversity. At each tree split, only a random subset of features is considered. The goal is to lower correlation ρ between trees.

The variance formula above explains the strategy. A forest may accept a modest increase in individual-tree bias if feature randomization materially decreases inter-tree correlation. The ensemble can then have lower overall error.

A random forest can therefore be thought of as

$$\boxed{\text{bootstrap data randomization} + \text{feature randomization} + \text{averaging}.}$$

15. Out-of-bag evaluation

Because approximately 36.8% of observations are omitted from each bootstrap sample, every observation can be predicted by the subset of models that did not train on it.

If $\mathcal{M}_i$ is the set of models for which observation i was out-of-bag, define

$$\hat{y}_i^{\text{OOB}} = \frac{1}{|\mathcal{M}_i|} \sum_{m \in \mathcal{M}_i} \hat{f}_m(x_i).$$

OOB predictions provide an internal estimate of generalization error for bootstrap ensembles, reducing the need for a separate validation split in some standard IID settings.

For time-series data, however, ordinary bootstrap/OOB logic can break temporal dependence and create look-ahead problems. Block bootstrap, stationary bootstrap, or explicitly time-ordered ensembles are needed when chronology matters.

16. General ensemble methods

Bagging is only one ensemble strategy.

16.1. Simple averaging

Given predictions $\hat{y}_m$,

$$\hat{y}_{\text{ens}} = \frac{1}{M} \sum_m \hat{y}_m.$$

This works surprisingly well when component models have comparable quality and partially uncorrelated errors.

16.2. Weighted averaging

Use

$$\boxed{\hat{y}_{\text{ens}} = \sum_{m=1}^M w_m \hat{y}_m, \quad \sum_m w_m = 1.}$$

Weights can be chosen manually or learned on validation/OOF predictions. Constraints such as $w_m \geq 0$ can improve stability.

16.3. Optimal linear combination and covariance

Suppose prediction errors have covariance matrix Σ . For unbiased estimators and weights summing to one, ensemble variance is

$$w^T \Sigma w.$$

The minimum-variance weights solve

$$\min_w w^T \Sigma w \quad \text{subject to} \quad \mathbf{1}^T w = 1.$$

Ignoring nonnegativity constraints,

$$w^* = \frac{\Sigma^{-1} \mathbf{1}}{\mathbf{1}^T \Sigma^{-1} \mathbf{1}}.$$

This makes the role of diversity explicit: the covariance structure matters, not just standalone model accuracy.

16.4. Stacking

Stacking trains a meta-model on base-model predictions:

$$\hat{y} = g(\hat{y}_1, \dots, \hat{y}_M).$$

To avoid leakage, the meta-model must be trained on **out-of-fold or out-of-sample base predictions**, not predictions from base models evaluated on the same observations used to fit them.

16.5. Bagging versus boosting

Bagging trains models largely independently and averages them, primarily reducing variance. Boosting trains models sequentially, with later learners focusing on residual mistakes of earlier learners, often reducing bias as well as variance. Random forest is a canonical bagging method; gradient boosting and XGBoost are canonical boosting methods.

17. How the methods relate mathematically

It is useful to classify regularizers by what object they constrain.

Method	What is constrained or randomized?	Main effect
Ridge	Parameter magnitude via L2	Smooth shrinkage; stability
LASSO	Parameter magnitude via L1	Sparse parameters; selection
Elastic Net	L1 + L2	Sparse but more stable/grouped
Data augmentation	Input distribution via label-preserving transforms	Invariance; larger effective sample
Noise injection	Inputs/activations/weights/gradients	Local robustness; smoothing
Early stopping	Optimization time	Suppresses late-fitting/weak directions
Sparse representation	Hidden/code activations	Selective latent representation
Dropout	Random activation/subnetwork masks	Co-adaptation control; stochastic ensemble
Bagging	Training samples + model averaging	Variance reduction
General ensemble	Prediction combination	Error diversification

Table 2: A unified view of the mechanisms.

18. Detailed worked comparison on one hypothetical problem

Suppose we have $n = 500$ observations and $p = 200$ standardized predictors. Predictors form correlated groups, only roughly 25 variables contain genuine signal, and a flexible neural network also overfits.

18.1. Ridge approach

Fit

$$\min_{\beta} \frac{1}{2n} \|y - X\beta\|^2 + \frac{\lambda}{2} \|\beta\|^2.$$

If the signal is diffuse across correlated variables, Ridge can keep them all with moderate coefficients. Validation determines λ .

18.2. LASSO approach

Fit

$$\min_{\beta} \frac{1}{2n} \|y - X\beta\|^2 + \lambda \|\beta\|_1.$$

Many coefficients become zero. If two genuine predictors are strongly correlated, LASSO may retain one and discard the other.

18.3. Elastic Net approach

Fit

$$\min_{\beta} \frac{1}{2n} \|y - X\beta\|^2 + \lambda \left(\alpha \|\beta\|_1 + \frac{1 - \alpha}{2} \|\beta\|^2 \right).$$

With $\alpha = 0.5$, for example, it can eliminate many weak variables while allowing correlated groups to coexist.

18.4. Neural-network approach

Suppose a network still overfits. We might combine several mechanisms:

$$L = L_{\text{data}} + \frac{\lambda}{2} \sum_l \|W_l\|_F^2,$$

train with modest dropout, perhaps inject domain-valid noise or perform augmentation, and use early stopping on a validation set. These mechanisms are complementary because they act in different ways.

18.5. Ensemble approach

Train several independently initialized networks or models on bootstrap/resampled subsets or different folds, then average:

$$\hat{y}_{\text{ens}} = \frac{1}{M} \sum_m \hat{y}_m.$$

If their errors are not perfectly correlated, prediction variance decreases.

19. Regularization in neural-network optimization

Consider a network with parameters θ . A training objective might be

$$L(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \ell_i(\theta) + \lambda_1 \|\theta\|_1 + \frac{\lambda_2}{2} \|\theta\|_2^2,$$

where in practice L1/L2 penalties may apply only to selected parameter tensors.

If dropout and data augmentation are also used, the stochastic objective is more accurately written

$$L(\theta) = \mathbb{E}_{T,m} \left[\frac{1}{B} \sum_{i \in \mathcal{B}} \ell(y_i, f_{\theta,m}(Tx_i)) \right] + \Omega(\theta).$$

Training approximates this expectation by randomly sampling mini-batches, transformations, and dropout masks. Early stopping then restricts how far optimization proceeds.

Thus modern neural-network training can contain multiple nested sources of randomness and regularization simultaneously.

20. Interaction effects and common mistakes

20.1. Too much regularization

Regularizers stack. Strong weight decay + high dropout + aggressive augmentation + early stopping can easily underfit. If both training and validation errors are high, adding more regularization is usually the wrong response.

20.2. Using augmentation that changes the label

An augmentation is only valid if its label-preservation assumption is credible. For example, horizontal flipping may be valid for many natural images but invalid for text, directional road signs, or asymmetric medical anatomy.

20.3. Applying LASSO without scaling

If feature x_1 is measured in dollars and x_2 in millions of dollars, their coefficient magnitudes are not comparable. L1/L2 penalties become unit-dependent. Standardization usually precedes regularized linear modeling.

20.4. Selecting hyperparameters on the test set

Choosing λ , α , dropout rate, augmentation strength, early-stopping epoch, or ensemble weights based on test performance leaks information from the test set. Use validation or nested/OOF procedures.

20.5. Random bootstrap for time series

Ordinary bagging assumes observations are exchangeable enough for bootstrap resampling. Randomly resampling individual rows of a time series can destroy serial structure and leak future regimes into training. Use time-aware resampling or time-ordered model ensembles.

20.6. Confusing parameter sparsity with dropout

A dropout mask temporarily removes activations during training. After training, those connections generally still exist. LASSO can make parameters structurally zero. These are different forms of sparsity.

21. Diagnostic patterns

21.1. High training performance, weak validation performance

This is classic overfitting. Candidate responses include stronger regularization, more valid augmentation, earlier stopping, reduced model capacity, or ensembles.

21.2. Training and validation both weak

This suggests underfitting, optimization failure, insufficient features, or excessive regularization. Reducing regularization may help; simply adding dropout or stronger penalties usually will not.

21.3. LASSO solutions unstable across folds

If selected features change drastically across folds, inspect multicollinearity and signal weakness. Elastic Net, stability selection, or grouped/domain-aware features may be more appropriate.

21.4. Ensemble gives little improvement

Likely causes include component models having strongly correlated errors, one model dominating in quality, or data leakage making individual estimates misleadingly optimistic. Diversity must be useful, not merely cosmetic.

22. Practical selection guide

Situation	Strong candidates	Why
Many correlated linear predictors	Ridge / Elastic Net	Stabilizes collinear directions
Need sparse interpretable linear model	LASSO / Elastic Net	Exact zeros
Correlated groups + desired sparsity	Elastic Net	Balances grouping and selection
Known label-preserving input symmetries	Data augmentation	Encodes invariance directly
Want local input robustness	Noise injection	Penalizes sensitivity around samples
Deep model starts overfitting late	Early stopping	Restricts optimization time
Hidden units co-adapt excessively	Dropout	Stochastic unit removal
High-variance base learner	Bagging	Prediction averaging reduces variance
Several good but different models	Weighted ensemble / stacking	Exploits complementary errors
Time-series prediction	Time-aware regularization and ensembles	Avoid IID resampling leakage

Table 3: A practical map from problem symptoms to candidate methods.

23. Connection to bias, variance, and model capacity

The methods can be positioned conceptually along three axes.

Explicit shrinkage reduces the effective parameter space:

Ridge, LASSO, Elastic Net.

Stochastic robustness makes the model succeed under perturbation:

augmentation, noise injection, dropout.

Procedure-level variance control modifies training duration or combines learners:

early stopping, bagging, ensembles.

No single method dominates universally because they regularize different failure modes.

24. Compact mathematical summary

Ridge

$$\hat{\beta} = \arg \min_{\beta} \frac{1}{2} \|y - X\beta\|^2 + \frac{\lambda}{2} \|\beta\|_2^2, \quad \hat{\beta} = (X^T X + \lambda I)^{-1} X^T y.$$

LASSO

$$\hat{\beta} = \arg \min_{\beta} \frac{1}{2} \|y - X\beta\|^2 + \lambda \|\beta\|_1,$$

$$\text{soft}(z, \lambda) = \text{sign}(z)(|z| - \lambda)_+.$$

Elastic Net

$$\hat{\beta} = \arg \min_{\beta} \frac{1}{2n} \|y - X\beta\|^2 + \lambda \left(\alpha \|\beta\|_1 + \frac{1 - \alpha}{2} \|\beta\|_2^2 \right).$$

Augmentation

$$L_{\text{aug}} = \frac{1}{n} \sum_i \mathbb{E}_T [\ell(y_i, f(Tx_i))].$$

Noise injection

$$\tilde{x} = x + \varepsilon, \quad L = \mathbb{E}_{\varepsilon} [\ell(y, f(x + \varepsilon))].$$

For linear least squares with isotropic Gaussian input noise:

$$\mathbb{E} \|y - (X + E)\beta\|^2 = \|y - X\beta\|^2 + n\sigma^2 \|\beta\|^2.$$

Early stopping

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t), \quad \text{retain } \theta_{t^*} \text{ at best validation checkpoint.}$$

Dropout

$$m_j \sim \text{Bernoulli}(q), \quad \tilde{h} = \frac{m \odot h}{q}, \quad \mathbb{E}[\tilde{h}] = h.$$

Bagging

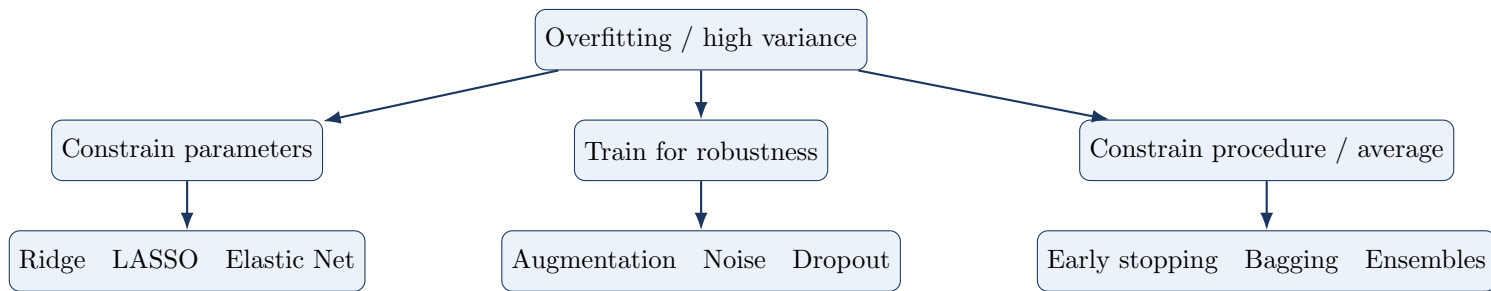
$$\hat{f}_{\text{bag}} = \frac{1}{M} \sum_m \hat{f}_m,$$

$$\text{Var}(\hat{f}_{\text{bag}}) = \sigma^2 \left[\rho + \frac{1 - \rho}{M} \right].$$

Weighted ensemble

$$\hat{y} = \sum_m w_m \hat{y}_m, \quad \sum_m w_m = 1.$$

25. Final conceptual map



A useful final summary is:

Ridge	: shrink everything smoothly,
LASSO	: shrink and delete parameters,
Elastic Net	: delete selectively while stabilizing correlations,
augmentation	: teach valid invariances,
noise	: teach local robustness,
early stopping	: do not optimize long enough to fit weak/noisy directions,
dropout	: train many stochastic subnetworks through shared weights,
bagging	: average unstable learners to lower variance,
ensembles	: combine complementary errors rather than trusting one model.

Suggested foundational references

For deeper study, standard sources include *The Elements of Statistical Learning* by Hastie, Tibshirani, and Friedman; *An Introduction to Statistical Learning* by James, Witten, Hastie, Tibshirani, and Taylor; *Deep Learning* by Goodfellow, Bengio, and Courville; and original papers on LASSO, Elastic Net, dropout, bagging, and random forests. The mathematical development in this note is intended to stand on its own while providing a map for those sources.